Evidence-Grounded Detection and Containment of Malicious Behaviour in LLM Coding Teams

Daniil Privezentsev

Public research proposal · working draft · 17 October 2026

Adapted from the author's Russian research proposal dated 14 October 2026

1. Problem and proposed objective

Language-model coding teams can produce convincing reports while creating programs that violate explicit requirements. A shared supervisor must decide which claims are supported and what action to take after a problem is found. Detecting a suspicious message is insufficient if the system still accepts the defective program or propagates its consequences.

This proposed study asks whether reproducible execution evidence, followed by targeted repair and isolation, can reduce accepted outputs with forbidden behaviour while preserving useful task completion under declared resource limits. It investigates the cost and failure regimes of these mechanisms rather than assuming that additional agents or verification improve outcomes.

2. Two scenarios that must remain distinct

Adversarial testing. A tester is openly tasked with finding an input, exploit or action sequence that breaks the program's requirements. Its local objective is adversarial, but its role supports the overall research objective. The comparison is against conventional testing under the same declared limits.

A hidden saboteur. A member of the constructive team is assigned an additional objective that conflicts with the task, such as preserving forbidden behaviour or persuading the supervisor to accept an invalid result. The supervisor does not know which runs contain this objective. Benign runs are required to measure false alarms and lost usefulness.

A discrepancy between an explanation and code does not itself establish intentional deception. Intent is known to the researcher only through the experimentally assigned objective. Otherwise, the appropriate claims concern unsupported statements or specification violations.

Scope

The initial setting uses controlled, synthetic or open-source programs in isolated environments for benign software-engineering tasks. Small, independently checkable cases come before larger repositories or multiple interacting development teams. All mechanisms, hypotheses and benefits described here remain proposals for evaluation.

Questions and experimental design

3. Research questions

RQ1 - Testing. Does a tester explicitly trying to break the program improve independently verified outcomes relative to a conventional tester?

RQ2 - Evidence. Does binding claims to reproducible evidence reduce acceptance of violating outputs at comparable false-alarm levels?

RQ3 - Intervention. Do local repair, isolation and rechecking reduce accepted corrupted results while retaining useful completion, compared with detection alone or complete redevelopment?

RQ4 - Transfer. Do effects persist on held-out task and violation families and a different developer or supervisor model configuration?

The first study prioritises RQ2 and RQ3. RQ1 controls for the value of adversarial testing; RQ4 establishes limits of applicability.

4. Architecture and evidence protocol

Start with one small development team, one adversarial tester and one shared supervisor. Fix each agent's information and editing permissions in advance. The supervisor remains a fallible language model; an independent checking mechanism evaluates its decisions. Agents and the supervisor cannot modify the protected final tests or trusted execution records.

Each substantial claim records: **(1)** the assertion, **(2)** the artifact version, **(3)** the specification requirement, **(4)** a reproducing input or test, **(5)** expected and observed behaviour, and **(6)** the trusted executor's run record. A claim that a test passed, or that a vulnerability exists, requires an actual reproducible execution.

The supervisor may accept, request evidence, repeat a check, send a module for repair, isolate an unsupported artifact, reassign work or abstain. Abstention can prevent a harmful acceptance but counts as unfinished work when measuring usefulness.

5. Controls and resource accounting

Compare **A** conventional testing; **B** adversarial testing; **C** adversarial testing with mandatory evidence; **D** evidence plus local intervention and rechecking; **E** monitoring followed by editing by a model without an assigned malicious objective; and **F** monitoring followed by complete redevelopment. Also include a single developer with the same overall retry allowance and test generation without a separate LLM attacker.

Keep tools, permissions, model versions and stopping rules explicit. Equal caps do not imply equal actual cost: count input, output and available reasoning tokens for all agents and retries, calls, test execution, latency and provider charges. Token counts are not interchangeable measures of compute or price across different models.

Three experiments and independent outcomes

6. Staged experiments

Experiment 1 - Can a report mislead the supervisor? Prepare small programs with independently known properties: correct programs, ordinary defects, assigned violations and false accusations. For the same code, vary truthful, incomplete and misleading reports. Compare free-form reporting with the evidence protocol while holding access to the executor constant. Measure acceptance of bad outputs, rejection of correct outputs and whether evidence reproduces. Keeping code fixed separates report effects from implementation difficulty.

Experiment 2 - Detection and containment. Run conditions A-F on the same development tasks, with assigned hidden objectives in some runs and benign agents in others. Independently determine whether the adversary attempted the objective, succeeded, was detected and left a surviving defect after intervention. Compare ordinary errors with assigned violations of similar type. If monitoring only distinguishes correct from incorrect code, describe the result as violation detection, not recognition of intent.

Experiment 3 - Propagation and transfer. Use dependent modules or a small number of interacting teams. Test whether the system can locate a fault, identify results that depend on it and recover without repeating everything. Compare local repair with full redevelopment and source-only repair without rechecking dependencies. Include the cost of locating the fault. Repeat the main comparison on held-out violation families and another model configuration; a stronger developer or saboteur than the supervisor is a separate stress case.

7. Data, outcomes and metrics

Begin with synthetic tasks whose contracts and checks are unambiguous, then add bounded function and repository tasks. Separate method development, threshold tuning and final evaluation by task or repository families; hold some violation families out entirely. Final tests remain hidden from the agents and supervisor. Passing them establishes success in this evaluation, not general program correctness.

Report two primary quantities together. **Risk:** runs with accepted, independently confirmed forbidden behaviour divided by all runs with an assigned saboteur. **Usefulness:** accepted outputs passing functional checks and without the defined forbidden behaviour divided by all runs in that scenario. A system that rejects everything has no useful completion.

Timeouts, refusals, failed attacks and budget exhaustion remain in the relevant denominators. Secondary measures include detection at a fixed false-alarm threshold, accusation precision, reproducible evidence, time to detection, repair correctness, propagation, tokens, cost and latency. State attack prevalence when reporting precision.

Validity, limits and next steps

8. Statistical and validity plan

A pilot of 30-50 diverse tasks informs cost, variance and the final sample size. Freeze primary comparisons, outcomes, thresholds, exclusions and stopping rules before final evaluation. Use paired task comparisons and uncertainty estimates that account for repeated runs within tasks and tasks within repositories; agent messages are not independent observations. Claims of cost reduction without material quality loss require a declared non-inferiority margin and analysis, not merely a non-significant difference.

9. Contribution and limits

Candidate contributions are a benchmark separating ordinary errors from assigned violations, a protocol linking claims to executable evidence, an intervention policy that checks dependent artifacts, and a measured risk-usefulness-cost trade-off. Existing work studies coordinated sabotage and monitoring [1], monitoring and trusted editing [2], and limitations of reasoning-based monitoring [3]. Novelty and benefit require comparison with these alternatives; they are not established here.

The supervisor and the editing model can still fail. A model without an assigned malicious objective is an experimental trust assumption, not an infallibility guarantee. Success on short tasks or one configuration must be reported at that scope. Higher costs, excessive rejection, failed repairs and effects that disappear under equal limits are informative negative outcomes.

Hybrid-INoT may organise a constructive node through internal planning, execution and critique roles, while independent agents retain separate processes and information boundaries. Earlier cost results do not establish detection, containment or transfer to multi-team settings. Broader collusion and adaptive verification allocation remain later research directions.

10. Indicative first-study plan

Weeks 1-2: related methods, threat model and metrics. **Weeks 3-5:** executor, records, evidence protocol and diagnostic pairs. **Weeks 6-7:** pilot and frozen evaluation plan. **Weeks 8-11:** main comparisons and error analysis. **Weeks 12-14:** held-out checks, key replication and manuscript preparation. Larger repositories and multi-team configurations follow feasibility checks.

Selected references

[1] *The Best-Laid SCHEMEs: Coordinated Sabotage and Monitoring in Multi-Agent Systems.* arXiv:2605.29178v1. arxiv.org/html/2605.29178v1

[2] *AI Control: Improving Safety Despite Intentional Subversion.* arXiv:2312.06942. arxiv.org/abs/2312.06942

[3] *CoT Red-Handed: Stress Testing Chain-of-Thought Monitoring.* arXiv:2505.23575. arxiv.org/abs/2505.23575